

第4章 多态性与虚函数

多态性也是 C++ 语言的重要特性之一。前面介绍的函数重载和运算符重载是 C++ 多态性的一种类型（即静态多态性）。本章将重点介绍 C++ 多态性的另外一种类型，也就是动态多态性。在 C++ 语言中，动态多态性主要是依靠虚函数来实现。在介绍多态性之前，先通过了解基类与派生类之间的兼容关系来体会为什么来引入多态性机制。

4.1 基类与派生类的对象兼容关系

在上一章中，我们学习了 C++ 的继承和派生技术。而且，我们知道派生类能继承基类的所有成员，包括数据成员和成员函数。但是这些基类的成员在派生类的访问属性与继承方式有关。一般情况下，程序设计中比较普遍使用的是公有继承方式。在公有继承方式下，派生类才是基类真正的子类型，它完整地继承了基类的功能。此时，基类与派生类的对象之间有一种兼容关系，即派生类中包含基类的成员。因此，可以将派生类某一个对象的值赋给基类对象；或者说，在用到基类对象时可以用其公有派生类的对象代替。这种兼容关系具体表现在以下几个方面：

1. 派生类对象可以向基类对象赋值。

可以用公有派生类对象向其基类对象赋值。假设 B 是基类 A 的公有派生类。如果采用 A 和 B 类分别构建了对象如下：

```
A AObj; //定义基类A对象AObj
```

```
B BObj; //定义类A的公用派生类 B的对象BObj
```

```
AObj=BObj; // 用派生类对象 BObj 向基类对象 AObj 赋值, 赋值时舍弃派生类自己的成员
```

需要注意：

- (1) 实际上，所谓赋值只是对数据成员赋值，对成员函数不存在赋值问题。
- (2) 这种兼容关系是单向的、不可逆的。也就是说，只能用派生类对象向其基类对象赋值，而不能用基类对象对其派生类对象赋值。为什么？同学们自己可以思考。
- (3) 同一基类的不同派生类对象之间也不能赋值。

2. 派生类对象可以替代基类对象向基类对象的引用进行赋值。

同上，假设 B 是基类 A 的公有派生类。如果采用 A 和 B 类分别构建了对象如下：

```
A AObj; //定义基类A对象AObj
```

```
B BObj; //定义类A的公用派生类 B的对象BObj
```

```
A& alias_a= AObj; //定义基类A对象的引用变量alias_a，并用AObj对其初始化；这时，引用变量alias_a是AObj的别名，它们共享同一段存储单元。
```

```
A& alias_a= BObj; // 用派生类对象向基类对象的引用变量alias_a初始化；但此时alias_a并不是BObj的别名，也不与 BObj共享同一段存储单元。它只是BObj中基类部分的别名，alias_a与BObj中基类部分共享同一段存储单元。
```

3. 如果函数的形参类型为基类对象或基类对象引，可采用派生类对象作实参

假设有一函数

```
void f1(A& alias_a) //形参是类A的对象的引用变量
{cout<<alias_a.num<<endl;}
```

函数的形参是类 A 的对象的引用变量，本来实参应该为 A 类的对象。由于派生类对象与基类对象赋值兼容。因此，在调用 f1() 函数时，可以用派生类 B 的对象 BObj 作实参：fun(b1)；输出类 B 的对象 BObj 的基类数据成员 num 的值。

4. 派生类对象的地址可以赋给指向基类对象的指针变量

或者说，指向基类对象的指针变量也可以指向派生类对象。假设定义一个基类 Student(学生)，再定义 Student 类的公有派生类 Graduate(研究生)，通过指向基类对象的指针变量来指向派生类对象，并用该指针变量输出基类对象的数据。

下面通过程序实例 4-1 来说明上述的含义。

实例 4-1，假设学生类 Student 是基类，采用公有继承方式派生了一个 Graduate 类，并在其中新增一个数据成员 pay。具体程序代码如下所示。

```
#include <iostream>
#include <string>
using namespace std;
class Employee
{
public :
    Employee(int, string, float );
    void display( );
private :
    int num;
    string name;
    float salary;
};

Employee::Employee(int n, string nam, float s) //定义构造函数
{
    num=n;
    name=nam;
    salary=s;
}

void Employee::display( ) //定义输出函数
{
    cout<<endl<<"num:"<<num<<endl;
    cout<<"name:"<<name<<endl;
    cout<<"salary:"<<salary<<endl;
}

class Intern:public Employee //声明公用派生类Intern
{
public :
    Intern(int, string ,float ,float );
    void display( );
private :
    float probation;
};
```

```

Intern::Intern(int n, string nam, float s, float p):Employee(n, nam, s), probation(p) { } //
定义构造函数

void Intern::display()    //定义输出函数,使用同名覆盖机制
{
    Employee::display();    //调用Employee类的display函数
    cout<<"probation:"<<probation<<endl;
}

int main()
{
    Employee emp1(1010, "Li", 15788);    //定义Employee类对象emp1
    Intern intern1(1011, "Wang", 3000, 6);    //定义Intern类对象intern1
    Employee *pt=&emp1; //定义指向Employee类对象的指针并指向emp1
    Intern *ptr=&intern1; //定义指向Intern类对象的指针并指向intern1
    pt->display( );    //调用emp1.display函数
    pt=&intern1;    //指针指向intern1
    pt->display( );    //调用emp1.display函数; 特别关注
    ptr->display();    //调用intern1.display函数
}

```

程序的输出结果:

```

num: 1010
name:Li
salary:15788
Num:1011
name:Wang
salary:3000
Num:1011
name:Wang
salary:3000
probation:6

```

通过分析程序及其结果,读者可能有点觉得意外。因为,在派生类中有两个同名的 display 成员函数,其中一个是从基类中继承来的,用来输出基类成员的数据,另外一个派生类新增的,用来输出基类和派生类数据。

在上述程序中:Employee *pt=&emp1; pt->display(); 这2条语句是调用 emp1.display 函数。即基类中的 display 成员函数。输出结果也证明如此。而 pt=&intern1; pt->display(); 根据同名覆盖的规则,这2条语句应该是调用 intern1.display 函数。即派生类中的 display 成员函数。其输出结果应该是先输出 num, name, salary, 然后再输出 probation 的值。而事实上,输出结果并没有输出 probation 的值。这是为什么呢?

问题在于 pt 是指向基类(Employee)类对象的指针变量,即使让它指向了 intern1,但实际上 pt 指向的是 intern1 中从基类继承的部分。也就是说,通过指向基类对象的指针,只能访问派生类中的基类成员,而不能访问派生类增加的成员。因此, pt->display() 调用的不是派生类 Graduate 对象所增加的 display 函数,而是基类的 display 函数,所以只输出研究生 intern1 的 num, name, salary 3 个数据。

当然,定义指向派生类对象的指针变量 ptr,使它指向 intern1,然后用 ptr->display() 调用派生类对象的 display 函数,此时可以输出 probation 的值。

通过本例可以得出如下结论:用指向基类对象的指针变量指向派生类对象是合法的、安

全的，不会出现编译上的错误。但在应用上，程序设计者往往有时希望通过使用基类指针能够调用基类和派生类对象的成员。是否可以实现此目标呢？答案是肯定的。这就是 C++ 语言拥有的第 4 大特性—动态多态性。

4.2 多态性与虚函数

4.2.1 多态性概念及类型

1. 多态性的概念

在 C++ 面向对象的程序设计中，所谓多态性 (polymorphism) 是指程序向不同对象发送同一个消息 (调用同名函数)，而这些不同对象在接收到同一消息时会产生不同行为 (实现不同功能)。也就是说，每个对象可以用自己的方式去响应同一消息。

如图 4-1 所示。假设有 Circle、star、Rect 和 Triangle 4 种类。在这些类中，分别都定义了成员函数 Draw()。如果希望在 main() 函数中，通过调用不同对象的 Draw()，即向这些不同对象发消息 (调用不同对象的成员函数 Draw())，能实现相应对象中 Draw() 函数的不同功能。这就是对象的多态性问题。

多态性机制想解决的问题是：当一个基类被继承为不同的派生类时，各派生类可以使用与基类成员相同的成员函数名。如果在运行时用同一个成员函数名调用类对象的成员函数，如何唯一地确定调用哪个对象的成员函数呢？而且，这些同一类族中的不同派生类，与基类成员同名的成员函数在不同的派生类中有不同的含义。因此，多态性机制就期望实现“一个接口，多种功能”。

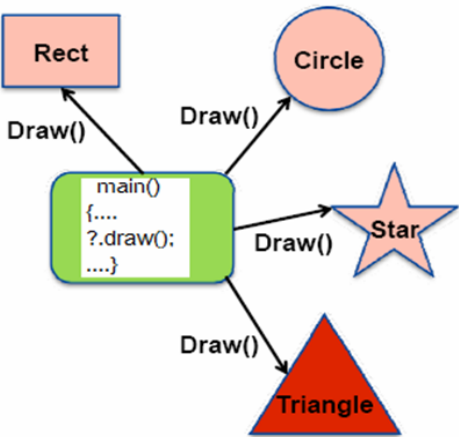


图 4-1 对象的多态性示意

多态性可以对不同对象使用同一的接口，既提高程序维护的可靠性和方便性。还能使得不同的对象的内部设计与外部使用接口分离，实现软件开发的分工协作。在 C++ 程序设计中，在不同的类中定义了其响应消息的具体方法 (公用成员函数)。那么使用这些类时，就不必考虑这些函数是如何具体实现的，只要调用函数发布消息即可。因此，多态性是面向对象程序设计的一个重要特征。

2. 多态性分类

在 C++ 语言中，多态性体现在许多方面。站在不同角度，分类方法也有所不同。下面主要从系统功能的角度看，多态性可分为 4 种类型。

- (1) 重载多态：函数或运算符重载；
- (2) 强制多态：数据或对象的类型强制转换；
- (3) 包含多态：类族中定义的同名成员函数的多态行为，主要通过虚函数来实现；
- (4) 参数多态：类模板实例化时的多态性，即实例化后的各个类都具有相同的操作。

但如果从多态性实现方式来分类，多态性分为两类：静态多态性和动态多态性。

(1) 静态多态性是通过函数的重载实现的(运算符重载实质上也是函数重载)，即同一函数名或运算符名称，通过重载能实现不同的功能。在程序编译时系统就能决定调用的是哪个函数，因此，静态多态性又称编译时的多态性。

(2) 动态多态性是在程序运行过程中才动态地确定操作所针对的对象。它又称运行时的多态性。动态多态性是通过虚函数(virtual function)实现的。

有关静态多态性，在运算符重载时做过介绍。本章重点学习 C++ 的动态多态性。

4.2.2 虚函数

1. 虚函数的作用

从类的继承与派生规则可以看出：在类的继承层次结构中，在不同的层次中可以出现名字相同、参数个数和类型都相同而功能不同的函数。编译系统按照同名覆盖的原则决定调用的对象。但也可以使用类限定符来指定调用那层类中的同名成员。但这种通过类限定符的方法来区分两个同名的函数，使用起来很不方便。良好程序设计架构追求的目标是：使用同一个调用形式，既能调用派生类又能调用基类的同名函数。

该目标基本思想：在程序中不是通过不同的对象名去调用不同派生层次中的同名函数，而是通过指针调用它们。例如，在程序实例 4-1 中，希望用同一个语句“`pt->display()`；”可以调用不同派生层次中的 `display()` 函数。只需在调用前给指针变量 `pt` 赋以不同的值(使之指向不同的类对象)即可，即所谓的动态多态性。

但从前面 4.1 节中程序实例来看，目前所学知识还无法实现该目标。要真正实现该目标，就需要进一步学习 C++ 中的虚函数技术。虚函数的作用是允许在派生类中重新定义与基类同名的函数，并且可以通过基类指针变量或引用来访问基类和派生类中的同名函数。

虚函数声明的一般形式为：

Virtual 函数类型 函数名称 (参数列表)；

例如：`virtual void display()`；

下面通过程序实例来说明虚函数是如何实现动态多态性的。

实例 4-2 在程序实例 4-1 基础上，作一点修改：在 `Employee` 类中声明 `display` 函数时，在最左面加一个关键字 `virtual`，即 `virtual void display()`；这样就把 `Employee` 类的

display() 函数声明为虚函数。试分析程序运行结果与实例 4-1 的差异。

```
#include <iostream>
#include <string>
using namespace std;
class Employee
{
public :
    Employee(int, string, float );
    virtual void display( );// display() 函数声明为虚函数
private :
    int num;
    string name;
    float salary;
};
Employee::Employee(int n, string nam, float s)
{
    num=n;
    name=nam;
    salary=s;
}
void Employee::display( ) //定义输出函数
{
    cout<<endl<<"num:"<<num<<endl;
    cout<<"name:"<<name<<endl;
    cout<<"salary:"<<salary<<endl;
}
class Intern:public Employee //声明公用派生类Intern
{
public :
    Intern(int, string ,float ,float );
    virtual void display( ); //display() 函数也声明为虚函数
private :
    float probation;
};
Intern::Intern(int n, string nam, float s, float p):Employee(n, nam, s), probation(p) { }
void Intern::display() //定义输出函数, 使用同名覆盖机制
{
    Employee::display(); //调用Employee类的display函数
    cout<<"probation:"<<probation<<endl;
}
int main()
{
    Employee emp1(1010, "Li", 15788); //定义Student类对象emp1
    Intern intern1(1011, "Wang", 3000, 6); //定义Intern类对象intern1
    Employee *pt=&emp1; //定义指向Student类对象的指针并指向emp1
    pt->display( ); //调用emp1.display函数
    pt=&intern1; //指针指向intern1
    pt->display( ); //调用intern1.display函数, 实现动态多态性
}
```

```
return 1;  
}
```

程序的输出结果:

```
num:1010  
name:Li  
salary:15788  
Num:1011  
name: Wang  
salary:3000  
probation:6 (这一项在实例 4-1 中没有输出的)
```

通过对比分析实例 4-1 和实例 4-2, 可以发现两者代码差异, 只是在程序实例 4-2 中作一点修改, 在 Employee 类中声明 display() 函数时, 在最左边加一个关键字 virtual, 这样就把 Employee 类的 display 函数声明为虚函数。程序其他部分都不改动。运行程序结果, 程序实例 4-2 就能输出成员 probation 的结果。证明, pt=&intern1; pt->display(); 是派生类中 display() 函数, 而不是基类中的 display() 函数。这样同一的函数调用语句, 实现不同的函数调用功能。这就是所谓的通过虚函数来实现动态多态性。也就是说, 由虚函数实现的动态多态性就是: 同一类族中不同类的对象, 对同一函数调用做出不同的响应。

下面对虚函数的使用方法总结如下:

(1) 在基类用 virtual 声明成员函数为虚函数。这样就可以在派生类中重新定义此函数, 为其赋予新的功能, 并能方便地被调用。在类外定义虚函数时, 不必再加 virtual。

(2) 在派生类中重新定义此函数, 要求函数名、函数类型、函数参数个数和类型全部与基类的虚函数相同, 并根据派生类的需要重新定义函数体。如果在派生类中没有对基类的虚函数重新定义, 则派生类简单地继承其直接基类的虚函数。另外, C++ 规定, 当一个成员函数被声明为虚函数后, 其派生类中的同名函数都自动成为虚函数。因此, 在派生类重新声明该虚函数时, 可以加 virtual, 也可以不加, 但习惯上一般在每一层声明该函数时都加 virtual, 使程序更加清晰。提问: 如果一个基类中有一个成员函数被声明为虚函数后, 在同一类族中派生类中是否可以再定义一个非 virtual 的、且与该虚函数具有相同的参数(包括个数和类型)和函数返回值类型的同名函数? 为什么? (读者自己考虑)

(3) 定义一个指向基类对象的指针变量, 并使它指向同一类族中需要调用该函数的对象。通过该指针变量调用此虚函数, 此时调用的就是指针变量指向的对象的同名函数。也就是说, 如果指针不断地指向同一类族中不同类的对象, 就能不断地调用这些对象中的同名函数。

另外, 下列几种函数不能声明为虚函数。

(1) 只能用 virtual 声明类的成员函数, 使它成为虚函数, 而不能将类外的普通函数声明为虚函数。因为虚函数的作用是允许在派生类中对基类的虚函数重新定义。显然, 它只能用于类的继承层次结构中。

(2) 类的静态成员函数不可以申明为虚函数; 因为静态成员函数属于类, 而不属于对象的属性。

(3) 内联函数不可以申明为虚函数。因为，内联函数在编译时就进行代码展开。

(4) 构造函数不可以为申明虚函数。因为构造函数调用时，对象还没有产生。但析构函数可以申明为虚函数，而且一般鼓励将类的析构函数尽可能申明为虚函数。

那么，一般在什么情况下使用虚函数呢？主要考虑以下几点：

(1) 首先看成员函数所在的类是否会作为基类。然后看成员函数在类的继承后有无可能被更改功能，如果希望更改其功能的，一般应该将它申明为虚函数。反之，如果成员函数在类被继承后功能不需修改，或派生类用不到该函数，则不要把它声明为虚函数。

(2) 考虑对成员函数的调用是通过对象名还是通过基类指针或引用去访问，如果是通过基类指针或引用去访问的，则应当声明为虚函数。为什么？（请读者考虑）。

(3) 有时在定义虚函数时，并不定义其函数体，即函数体是空的。它的作用只是定义了一个虚函数名，具体功能留给派生类去添加，即纯虚函数。这一点将在后面章节中详细讨论。

但是，是不是类中虚函数申明得越多越好呢？这需要均衡考虑。因为使用虚函数时，系统要有一定的空间开销。这是因为当一个类带有虚函数时，编译系统会为该构造一个虚函数表(virtual function table, 简称 vtable)，它是一个指针数组，存放每个虚函数的入口地址。例如，对于程序实例 4-2，编译系统会产生如图 4-2 所示的虚函数表。通过这个虚函数表，程序执行时，系统就能确定调用的是哪个类对象中的函数。这种在程序执行时把虚函数和类对象“绑定”在一起的过程称为动态关联(dynamic binding)。由于动态关联是在编译以后的运行阶段进行的，因此也称为滞后关联(late binding)。在运行阶段，指针可以先后指向不同的类对象，从而调用同一类族中不同类的虚函数。

前面所提到的函数重载和通过对象名调用的虚函数，在编译时即可确定其调用的虚函数属于哪一个类，其过程称为静态关联(static binding)，由于是在运行前进行关联的，故又称为早期关联(early binding)。函数重载属静态关联。

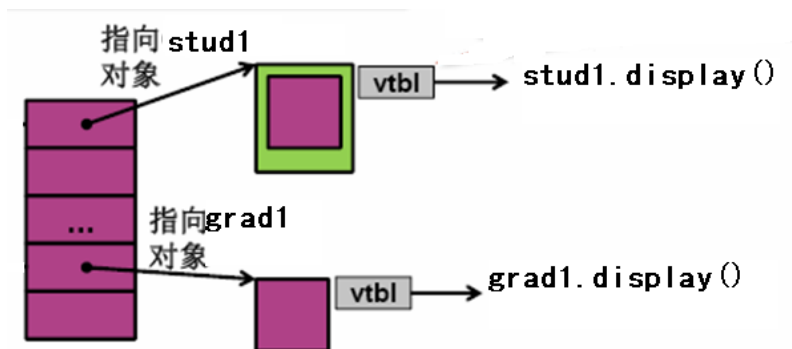


图 4-2 虚函数表结构

但系统在进行动态关联时的时间开销是很少的，因此，多态性是高效的。

另外，在理解动态多态性时，需要深刻体会下列概念的差异。

(1) 是否采用指针调用族类函数就是多态性？

例如，有时在基类中定义的非虚函数会在派生类中被重新定义(如例 4-1 中的 display() 函数)，如果用基类指针调用该成员函数，则系统会调用对象中基类部分的成员函数；如果

用派生类指针调用该成员函数，则系统会调用派生类对象中的成员函数，这并不是多态性行为(使用的是不同类型的指针)，没有用到虚函数的功能。

(2) 虚函数和函数重载的区别

以前介绍的函数重载处理的是同一作用域中的同名函数问题，而虚函数处理的是不同派生层次上的同名函数问题，两者可以归纳为如下几点。

(1) 重载函数的函数名称相同，参数不同；重载函数是在作用域相同的区域里定义的不同名字的不同函数；而虚函数的函数原型完全一致，体现在基类和派生类的类层次结构中。

(2) 重载函数可以是成员函数或友元函数或一般函数，而虚函数只能是成员函数。

(3) 调用重载函数以所传递参数序列的差别作为调用不同函数的依据；虚函数则根据对象的不同调用不同类的虚函数。

(4) 重载函数在编译时表现出多态性，是静态联编（早期绑定）；而虚函数在运行时表现出多态性，是动态联编（晚期绑定），动态联编是 C++ 的精髓。

下面来列举一个程序实例来说明通过虚函数实现动态多态性，对程序结构的统一框架设计的好处。重点分析 main() 的循环语句。同时体会静态多态性和动态多态性的区别。

实例 4-3 采用多态性机制，对 2 种不同类的对象数据进行统一调用。

```
#include <iostream>
#include "cylinder.h"
using namespace std;
int main()
{
    Circle * arrMix[15];
    for (int i = 0; i < 10; i++) //前10个为Cylinder对象
        arrMix[i] = new Cylinder(0, 0, i + 1, i + 1);
    for (int i = 10; i < 15; i++) //后5个为Circle对象
        arrMix[i] = new Circle(0, 0, i + 1);
    for(int i = 0; i < 15; i++)
    {
        cout << arrMix[i]->area() << endl;
        delete arrMix[i];
        arrMix[i] = NULL;
    }
    return 0;
}

/*下面是cylinder.h的定义*/
#ifndef __POINT__
#define __POINT__
#include <iostream>
using namespace std;
class Point
```

```

{
private:
    float x;
    float y;
public:
    Point(float xx = 0, float yy = 0):x(xx),y(yy) {}
    void setX(float xx) { x = xx;}
    void setY(float yy) { y = yy;}
    float getX() const { return x; }
    float getY() const { return y; }
    friend ostream& operator <<(ostream& out, const Point& p);
    //virtual float area( ) const = 0;           //定义纯虚函数
};
ostream& operator <<(ostream& out, const Point& p)
{
    out<<"("<<p.getX()<<"", "<<p.getY()<<"")"<<endl;
    return out;
}
#endif

```

上述程序运行结果如下：

```

C:\Windows\system32\cmd.exe
Cylinder area: Circle area: 12.5664
Cylinder area: Circle area: 50.2654
Cylinder area: Circle area: 113.097
Cylinder area: Circle area: 201.062
Cylinder area: Circle area: 314.159
Cylinder area: Circle area: 452.389
Cylinder area: Circle area: 615.752
Cylinder area: Circle area: 804.247
Cylinder area: Circle area: 1017.88
Cylinder area: Circle area: 1256.64
Circle area: 380.132
Circle area: 452.389
Circle area: 530.929
Circle area: 615.752
Circle area: 706.858
请按任意键继续. . .

```

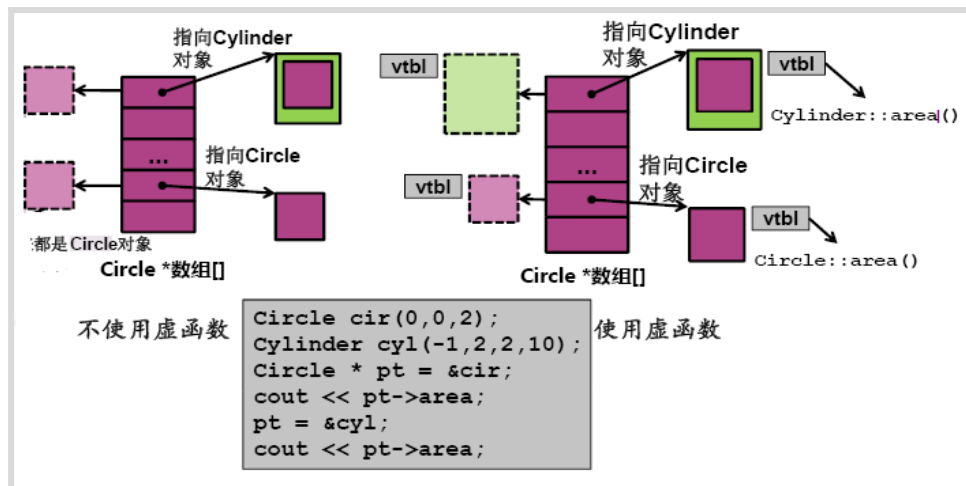


图 4-3 程序实例中虚函数调用关系

4.2.3 虚析构造函数

析构函数的作用是在对象撤销之前做必要的“清理现场”的工作。当派生类的对象从内存中撤销时，一般先调用派生类的析构函数，然后再调用基类的析构函数。

但是，如果用 new 运算符建立了临时对象，若基类中有析构函数，并且定义了一个指向该基类的指针变量。在程序用带指针参数的 delete 运算符撤销对象时，会发生下列情况：系统会只执行基类的析构函数，而不执行派生类的析构函数。如果希望能执行派生类的析构函数，则要求将基类的析构函数声明为虚析构函数。此时，对象析构时执行顺序为：先调用了派生类的析构函数，再调用了基类的析构函数。下面结合程序实例 4-4 来说明虚析构函数的使用和特点。

实例 4-4 虚析构函数的应用方法举例

```
#include <iostream>
#include <string>
using namespace std;
class Employee //声明Employee类
{
public :
    Employee(int, string, float );
    //这里直接使用了正确的写法，定义虚析构函数
    virtual ~Employee() {cout<<"executing employee destructor"<<endl;} //定义类析构函数
    virtual void display( );
private :
    int num;
    string name;
    float salary;
};
Employee::Employee(int n, string nam, float s) //定义构造函数
{
    num=n;
```

```

        name=nam;
        salary=s;
    }
    void Employee::display( ) //定义输出函数
    {
        cout<<endl<<"num:"<<num<<endl;
        cout<<"name:"<<name<<endl;
        cout<<"salary:"<<salary<<endl;
    }
    class Intern:public Employee //声明公用派生类Intern
    {
    public :
        Intern(int, string ,float ,float ) ;
        virtual ~Intern( ) {cout<<"executing intern destructor"<<endl;} //定义类析构函数
        virtual void display( );
    private :
        float probation;
    };
    Intern::Intern(int n, string nam,float s,float p):Employee(n, nam, s),    probation(p) { }
    //定义构造函数
    void Intern::display() //定义输出函数, 使用同名覆盖机制
    {
        Employee::display(); //调用Employee类的display函数
        cout<<"probation:"<<probation<<endl;
    }
    int main()
    {
        Employee *p=new Intern(20130023, "Jaime", 1900, 3); //用new开辟动态存储空间
        delete p;    //用delete释放动态存储空间
        return(0);
    }

```

p 是指向基类的指针变量，指向 new 开辟的动态存储空间，希望用 delete 释放 p 所指向的空间。但运行结果为：executing employee destructor

此结果表示只执行了基类 employee 析构函数，而没有执行派生类 graduate 的析构函数。如果希望能执行派生类 graduate 析构函数，就必须将基类的析构函数声明为虚析构函数。如下所示。virtual ~employee() {cout<<" executing employee destructor" <<endl;}

通过上述分析，对虚析构函数使用可以总结如下 2 点规则。

(1) 当基类的析构函数为虚函数时，无论指针指的是同一类族中的哪一个类对象，系统会采用动态关联，调用相应的析构函数，对该对象进行清理工作。

(2) 如果将基类的析构函数声明为虚函数时，由该基类所派生的所有派生类的析构函数也都自动成为虚函数。因此，良好的编程素养是即使基类并不需要析构函数，也显式地定义一个函数体为空的虚析构函数，以保证在撤销动态分配空间时能得到正确的处理。

4.2.4 多重继承中的虚函数

虚函数具有传递性，具有虚函数的基类指针可以指向它的派生类，也可以指向它的派生类的派生类，它们都具有动态特性。为了实现多重继承中虚函数的传递性，在派生时，需要采用 public 继承方式，否则无法实现虚函数特性。另外，多重继承中，需要采用虚基类 (virtual) 的继承关系。

实例 4-5，为了综合说明虚基类和虚函数的应用，下面列举了一个多重继承下的虚函数使用情况。在该应用中实现了一个公共基类 CPerson。CStudent 和 CTeacher 采用了公有继承虚基类的方式。CTeacherAndStudent 类则是一个多继承类，它属于 CPerson 的二级派生类。对所有这些类来说，Show() 函数都是虚函数，具有虚特性。

```
//filename:example_4_5_student.h
#ifndef __EXAMPLE_4_5_STUDENT_H__
#define __EXAMPLE_4_5_STUDENT_H__
#include <iostream>
using namespace std;

class CPerson
{
public:
    CPerson(char *lpszName, char *lpszSex);
    virtual void Show() const;
protected:
    char *m_lpszName;
    char *m_lpszSex;
};

class CStudent : virtual public CPerson
{
public:
    CStudent(char *lpszName, char *lpszSex, int iNumber);
    void Show() const;
protected:
    int m_iNumber;
};

class CTeacher : virtual public CPerson
{
public:
    CTeacher(char *lpszName, char *lpszSex, double dSalary);
    void Show() const;
protected:
    double m_dSalary;
```

```

};

class CTeacherAndStudent : public CStudent, public CTeacher
{
public:
    CTeacherAndStudent(char *lpszName, char *lpszSex, double dSalary, int iNumber);

    void Show() const;
};

void TestReference(const CPerson &rCPerson);

#endif

//FILENAME:EXAMPLE_4_5_student.cpp
#include "example_4_5_student.h"

CPerson::CPerson(char *lpszName, char *lpszSex)
{
    m_lpszName = lpszName;
    m_lpszSex = lpszSex;
}

void CPerson::Show() const
{
    cout << m_lpszName << "|" << m_lpszSex << endl;
}

CStudent::CStudent(char *lpszName, //姓名
    char *lpszSex, //性别
    int iNumber //学号
) : CPerson(lpszName, lpszSex)
{
    m_iNumber = iNumber;
}

//虚函数的具体实现
void CStudent::Show() const
{
    cout << m_lpszName << "|" << m_lpszSex << "|" << m_iNumber << endl;
}

CTeacher::CTeacher(char *lpszName, char *lpszSex, double dSalary) : CPerson(lpszName,
lpszSex)

```

```

{
    m_dSalary = dSalary;
}

void CTeacher::Show() const
{
    cout << m_lpszName << "|" << m_lpszSex << "|" << m_dSalary << endl;
}

CTeacherAndStudent::CTeacherAndStudent(char *lpszName,    char *lpszSex,    double
dSalary, int iNumber ) : CTeacher(lpszName, lpszSex, dSalary), CStudent(lpszName, lpszSex,
iNumber), CPerson(lpszName, lpszSex)
{ }

void CTeacherAndStudent::Show() const
{
    cout << m_lpszName << "|" << m_lpszSex << "|" << m_iNumber << "|" << m_dSalary << endl;
}

void TestReference(const CPerson &rCPerson)
{
    rCPerson.Show();
}

//FEILENAME MAIN.CPP
#include "example_4_5_student.h"
void main()
{
    CPerson oCPerson("李明", "男");
    CStudent oCStudent("王磊", "男", 20050101);
    CTeacher oCTeacher("张楚", "女", 5000);
    //初始化派生类对象
    CTeacherAndStudent oCTeacherAndStudent("赵晗", "女", 400, 20050202);

    cout<<"\n1: 以CPerson方式调用----"<<endl;
    TestReference(oCPerson);
    TestReference(oCStudent);
    TestReference(oCTeacher);
    TestReference(oCTeacherAndStudent);
    cout<<"\n2: 以CPerson指针调用----"<<endl;
    CPerson *pCPerson;
    pCPerson = &oCPerson;
    pCPerson->Show(); //调用pCPerson类中的Show()
    pCPerson = &oCStudent; //指向另一对象

```



```

pCPerson->Show(); //调用CStudent中的Show()
pCPerson = &oCTeacher; //指向CTeacher对象
pCPerson->Show(); //调用CTeacher中的Show()
pCPerson = &oCTeacherAndStudent; //指向派生类CTeacherAndStudent
pCPerson->Show(); //调用CTeacherAndStudent中的Show()

cout<<"\n3:以不同类指针调用----"<<endl;
CStudent *pCStudent = &oCTeacherAndStudent;
pCStudent->Show(); //调用pCStudent类中的Show()
CTeacher *pCTeacher = &oCTeacherAndStudent;
pCTeacher->Show(); //调用pCTeacher类中的Show()

system("pause");
}

```

4.3 纯虚函数与抽象类

4.3.1 纯虚函数的概念

前面学习了虚函数的概念，下面进一步介绍“纯虚函数”的概念。所谓纯虚函数是指没有函数体的虚函数，即纯虚函数是在声明虚函数时被“初始化”为0的函数。因此，声明纯虚函数的一般形式如下。

```
virtual 函数类型 函数名 (参数表列)=0;
```

例如：virtual float area()const =0; //纯虚函数

从申明格式可以看出，纯虚函数具有如下特点。

- ①纯虚函数没有函数体；
- ②最后面的“=0”并不表示函数返回值为0，它只起形式上的作用，告诉编译系统“这是纯虚函数”；
- ③这是一个声明语句，最后应有分号。

纯虚函数只有函数的名字而不具备函数的功能，不能被调用。因此，一般情况下，在基类中申明一个纯虚函数，它只是通知编译系统：“在这里声明一个虚函数，留待派生类中定义”。在派生类中对此函数进行了定义后，它才能具备函数的功能，可被调用。

例如，假设有一个基类Point，并且有直接派生类Circle和间接派生类Cylinder。对于基类Point，实际上没有必要有求面积的area()函数，因为“点”是没有面积的。但对于直接派生类Circle和间接派生类Cylinder来说，都需要有area函数，而且这两个area()函数的功能不同，一个是求圆面积，一个是求圆柱体表面积。

按照前面学习的虚函数技术，可以在基类Point中加一个area函数，并声明为纯虚函数(pure virtual function):virtual float area()const=0;

其实，在基类中并不使用这个函数，可以不写出这种无意义的函数体，只给出函数的原型。然后，分别在直接派生类 Circle 和间接派生类 Cylinder 中，再对函数 area() 定义不同的计算功能。

综上所述，纯虚函数的作用是在基类中为其派生类保留一个函数的名字，以便派生类根据需要对它进行定义。因为如果在基类中没有保留函数名字，则无法实现多态性。如果在一个类中声明了纯虚函数，而在其派生类中没有对该函数定义，则该虚函数在派生类中仍然为纯虚函数。

4.3.2 抽象类

在 C++ 等面向对象程序设计中，有一些类是不用来生成对象的。定义它们的目的是给用户提供一种基本类型，使得用户在这种基本类型的基础上，根据用户需要，再分别定义出功能各异的派生类。然后，用这些派生类去构建对象。

于是，C++ 又引入一个新概念——抽象类 (abstract class)。所谓抽象类是不用来定义对象而只作为一种基本类型用作继承的类。因为抽象类常用作基类继承，所以也称为抽象基类 (abstract base class)。

那么，哪些类是抽象基类呢？凡是包含纯虚函数的类都是抽象类。因为纯虚函数是不能被调用的，包含纯虚函数的类是无法建立对象的。

抽象类的作用是作为一个类族的共同基类，或者说，为一个类族提供一个公共接口。因为虽然抽象类不能定义对象 (或者说抽象类不能实例化)，但是可以定义指向抽象类数据的指针变量。当派生类成为具体类之后，就可以用这种指针指向派生类对象，然后通过该指针调用虚函数，实现多态性的操作。

如果在抽象类所派生出的新类中，对基类的所有纯虚函数进行了定义，那么这些函数就被赋予了功能，可以被调用。这个派生类就不是抽象类，而是可以用来定义对象的具体类 (concrete class)。如果在派生类中没有对所有纯虚函数进行定义，则此派生类仍然是抽象类，不能用来定义对象。因此，一个类层次结构中也可不包含任何抽象类，每层都是具体类。但是，在实际面向对象软件系统，其层次结构的顶部是一个抽象类，甚至顶部有好几层都是抽象类。

4.3.3 应用实例

为了进一步体会多态性、虚函数和纯虚函数，以及抽象类的概念，下面列举一个更加全面的程序实例来介绍说明。

实例 4-6 假设有一个类 Point (点)、以此为基础，派生出直接派生类 circle (圆) 类，间接派生类 Cylinder (圆柱体) 类的层次结构。为了引入纯虚函数和抽象基类等概念。类的层次结构的顶层是抽象基类 Shape (形状)，这样将类 Point 也作为类 Shape 的直接派生类。

程序中类之间的层次关系和属性如图 4-4 所示。

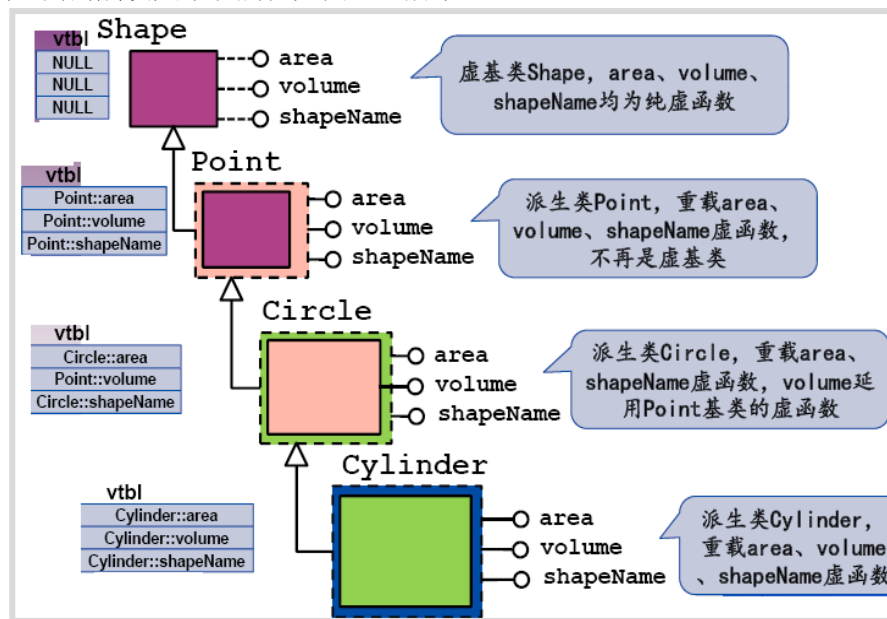


图 4-4 程序实例 4-6 中的类层次关系和属性

```

//shape.h
#ifndef __SHAPE__
#define __SHAPE__
#include <iostream>
using namespace std;
class Shape //声明抽象基类Shape
{
public:
    virtual float area() const {return 0.0;} //虚函数
    virtual float volume() const {return 0.0;} //虚函数
    virtual void shapeName() const =0; //纯虚函数
};
#endif

//point.h
#ifndef __POINT__
#define __POINT__
#include "shape.h"
class Point:public Shape //Point是Shape的公用派生类
{public:
    Point(float=0,float=0);
    void setPoint(float ,float ); float getX() const {return x;} float getY() const {return y;}

    virtual void shapeName() const {cout<<"Point:";} //对虚函数进行再定义
    friend ostream & operator <<(ostream &, const Point &);
protected:
    float x, y;
};
  
```

```

};
Point::Point(float a, float b)
{
    x=a;y=b;
}
void Point::setPoint(float a, float b)
{
    x=a;y=b;
}
ostream & operator <<(ostream &output, const Point &p)
{
    output<<"["<<p.x<<" "<<p.y<<""]";
    return output;
}
#endif

//circle.h
#ifndef __CIRCLE__
#define __CIRCLE__
#include "point.h"
class Circle:public Point //声明Circle类
{
public:
    Circle(float x=0, float y=0, float r=0);
    void setRadius(float );
    float getRadius( )const;
    virtual float area( )const;
    virtual void shapeName( )const {cout<<"Circle:";} //对虚函数进行再定义
    friend ostream &operator <<(ostream &, const Circle &);
protected :
    float radius;
};

Circle::Circle(float a, float b, float r):Point(a, b), radius(r) {}
void Circle::setRadius(float r){ radius = r; }
float Circle::getRadius( )const {return radius;}
float Circle::area( )const {return 3.14159*radius*radius;}
ostream &operator <<(ostream &output, const Circle &c)
{
    output<<"["<<c.x<<" "<<c.y<<""]", r="<<c.radius;
    return output;
}
#endif

//cylinder.h
#ifndef __CYLINDER__
#define __CYLINDER__
#include "circle.h"

```

```

class Cylinder:public Circle //声明Cylinder类
{
public:
    Cylinder (float x=0,float y=0,float r=0,float h=0);
    void setHeight(float);
    virtual float area( )const;
    virtual float volume( )const;
    virtual void shapeName( )const {cout<<"Cylinder:";} //对虚函数进行再定义
    friend ostream& operator <<(ostream&,const Cylinder&);
protected:
    float height;
};

Cylinder::Cylinder(float a,float b,float r,float h):Circle(a,b,r),height(h){}
void Cylinder::setHeight(float h){height=h;}
float Cylinder::area( )const
{
    return 2*Circle::area( )+2*3.14159*radius*height;
}
float Cylinder::volume( )const{return Circle::area( )*height;}
ostream &operator <<(ostream &output,const Cylinder& cy)
{
    output<<"["<<cy.x<<","<<cy.y<<"], r="<<cy.radius<<"," h="<<cy.height;
    return output;
}
#endif

//smain.cpp
include <iostream>
#include "cylinder.h"
using namespace std;

int main( )
{
    Point point(3.2,4.5); //建立Point类对象point
    Circle circle(2.4,1.2,5.6); //建立Circle类对象circle
    Cylinder cylinder(3.5,6.4,5.2,10.5); //建立Cylinder类对象cylinder
    point.shapeName(); //静态关联
    cout<<point<<endl;
    circle.shapeName(); //静态关联
    cout<<circle<<endl;
    cylinder.shapeName(); //静态关联
    cout<<cylinder<<endl<<endl;
    Shape *pt; //定义基类指针
    pt=&point; //指针指向Point类对象
    pt->shapeName( ); //动态关联
}

```

```

cout<<"x"<<point.getX()<<"y"<<point.getY()<<endl;
cout<<"area"<<pt->area()<<endl;
cout<<"volume"<<pt->volume()<<endl<<endl;
pt=&circle;           //指针指向Circle类对象
pt->shapeName();       //动态关联
cout<<"x"<<circle.getX()<<"y"<<circle.getY()<<endl;
cout<<"area"<<pt->area()<<endl;
cout<<"volume"<<pt->volume()<<endl<<endl;
pt=&cylinder;         //指针指向Cylinder类对象
pt->shapeName();       //动态关联
cout<<"x"<<cylinder.getX()<<"y"<<cylinder.getY()<<endl;
cout<<"area"<<pt->area()<<endl;
cout<<"volume"<<pt->volume()<<endl<<endl;
return 0;
}

```

程序运行结果如下。

请读者对照程序分析。

Point:[3.2,4.5] (Point 类对象 point 的数据:点的坐标)

Circle:[2.4,1.2], r=5.6 (Circle 类对象 circle 的数据:圆心和半径)

Cylinder:[3.5,6.4], r=5.2, h=10.5 (Cylinder 类对象 cylinder 的数据: 圆心、半径和高)

Point:x=3.2,y=4.5 (输出 Point 类对象 point 的数据:点的坐标)

area=0 (点的面积)

volume=0 (点的体积)

Circle:x=2.4,y=1.2 (输出 Circle 类对象 circle 的数据:圆心坐标)

area=98.5203 (圆的面积)

volume=0 (圆的体积)

Cylinder:x=3.5,y=6.4 (输出 Cylinder 类对象 cylinder 的数据:圆心坐标)

area=512.959 (圆柱体的表面积)

volume=891.96 (圆柱的体积)

通过分析上述程序,可以进一步明确以下结论。

(1) 一个基类如果包含一个或一个以上纯虚函数,就是抽象基类。抽象基类不能也不必要定义对象。抽象基类是用来定义本类族的公共接口的,或者说,从同一基类派生出的多个类有同一接口。因此,抽象基类体现了本类族中各类的共性,把各类中共有的成员函数集中在抽象基类中声明。

(2) 抽象基类与普通基类不同,它一般并不是现实存在对象的抽象(例如圆形(Circle)就是许多实际圆的抽象),它可以没有任何物理上的或其他实际意义方面的含义。

(3) 在类的层次结构中,顶层或最上面的几层可以是抽象基类。如果在基类声明了虚函数,则在派生类中凡是与该函数有相同的函数名、函数类型、参数个数和类型的函数,均为虚函数(不论在派生类中是否用 virtual 声明)。

4.4 综合程序举例

在前面一章的综合程序应用的基础上，对“犀利人事管理系统”进行功能扩展和修改。要求如下：

(1) 在 4 个类中分别定义一个同名函数 (PAY()) 来计算 4 类人员的工资；并要求在 main() 函数中，采用指向基类指针变量来调用这 4 个类中的 PAY() 函数。

(2) 在公司员工的要求下，公司对员工工资每隔 1 年就调级。4 类岗位的调级方法不同，具体上浮多少工资由程序员来确定。建议：采用升级函数 promote (参数) 来实现，考虑不同类中 promote (参数) 的功能不同，因此，可以在基类中将 promote 申明为虚函数，用多态性来实现程序功能。

本章小结

本章重点是学习如何通过虚函数来实现动态多态性。多态性是 C++ 语言的又一重要特性。函数和运算符重载等主要体现了 C++ 的静态多态性。而虚函数是实现动态多态性。使用虚函数可以提高程序的可扩充性，使得类的声明与类的使用分离。例如，开发商设计了各种各样的类，但不向用户提供源代码，用户可以不知道类是怎样声明的，但是可以使用这些类来派生出自己的类。利用虚函数和多态性，程序员的注意力集中在处理普遍性，而让执行环境处理特殊性。多态性把操作的细节留给类的设计者(他们多为专业人员)去完成，而让程序人员(类的使用者)只需要做一些宏观性的工作，告诉系统做什么，而不必考虑怎么做，极大地简化了应用程序的开发工作。

另外，在虚函数概念的基础上，进一步引入了纯虚函数和抽象基类的概念，充分地体现了 C++ 的抽象性特点。

课后习题

1. 阅读 4.4 节综合程序代码，梳理出程序结构，结合本章的知识点，体会该程序应该了哪些知识点？如何使用的？
2. 在 C++ 语言中，能否声明虚构造函数？为什么？能否声明虚析构函数？有何用途？
3. 什么叫做抽象类？抽象类有何作用？抽象类的派生类是否一定要给出纯虚函数？
4. 分析虚函数与函数重载区别？
5. 分析程序的运行结果

a)

```
#include<iostream>
using namespace std;
```

```

class B
{
    public:
        B(int i) { b=i+50;show;}
        B() { }
        virtual void show()
        {
            cout<<"B::show() called. "<<b<<endl;
        }
    protected:
        int b;
};

class D: public B
{
    public:
        D(int i) { d=i+100; show(); }
        D() { }
        void show()
        {
            cout<<" D::show() called. "<<d<<endl;
        }
    protected:
        int d;
};

void main()
{
    D d1(108);
}

b)
#include<iostream>
using namespace std;
class A
{
    public:
        A() { ver=' A' ; }
        void print() { cout<<" The A version : "<<ver<<endl; }
    protected:
        char ver;
};

class D1: public A
{
    public:
        D1(int number) { info=number; ver=' 1' ; }
        void print()

```



```

        { cout<<" The D1  info:  "<<info<<" version" <<ver<<endl;  }
private:
    int info;
};
class D2:public  A
{
public:
    D2(int  number)  {  info=number;  }
    void  print()
    { cout<<" The  D2  info:  "<<info<<" version" <<ver<<endl;  }
private:
    int  info;
};
class D3:public  D1
{
public:
    D3(int  number):D1(number)
    {
        info=number;
        ver=' 3' ;
    }
    void  print()
    { cout<<" The  D3  info:  "<<info<<" version" <<ver<<endl;  }
private:
    int info;
};
void print_info(A  *p)
{
    p->print();
}
void main( )
{
    A a;
    D1  d1(4);
    D2  d2(100);
    D3  d3(-25);
    print_info(&a);
    print_info(&d1);
    print_info(&d2);
    print_info(&d3);
}

```

6. 声明一个哺乳动物 Mammal 类，再由此派生出狗 Dog 类，二者都定义 Speak()成员函数，基类中定义为虚函数。声明一个 Dog 类的对象，调用 Speak() 函数，观察运行结果。

7. 应用抽象类，求圆、圆内接正方形和圆外切正方形的面积和周长。
8. 声明一个 Shape 抽象类，在此基础上派生出 Rectangle 和 Circle 类，二者都有 GetArea() 函数计算对象的面积，GetPerim() 函数计算对象的周长。给定部分程序代码，根据题意要求补全基类代码，完成整个程序？

```
#include<iostream>
using namespace std;
class Shape
{
public:
    Shape() {}
    ~Shape() {}
    virtual float GetArea()=0;
    virtual float GetPerim()=0;
};
```